MLOps Lab Assignment: ML Pipelines, FastAPI and Docker

Sessions 7, 8 and 9 - Report

Course: MLOps: From Machine Learning to Production AI Systems

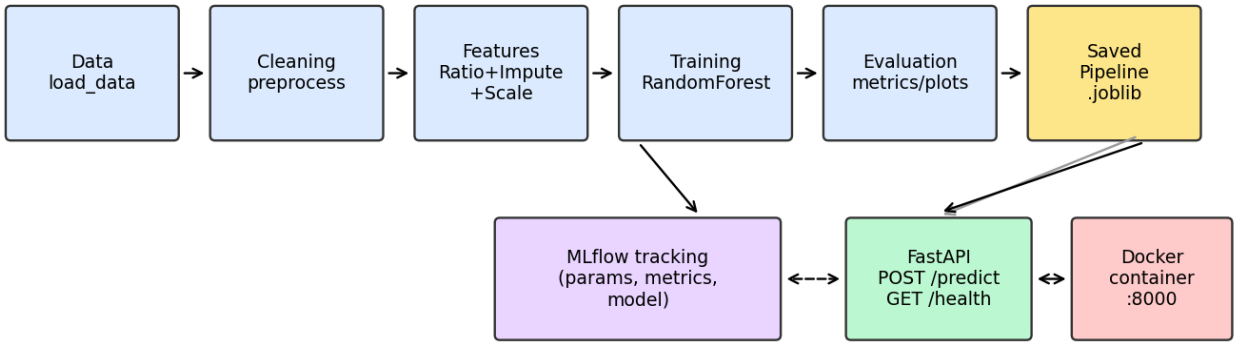
Evidence key. **VERIFIED** = executed when this project was generated (outputs stored under *evidence/* and *reports/*). **RUN ON YOUR MACHINE** = requires Docker or a browser/Postman, which were not available in the generation environment; the exact command that produces the evidence is given, and *evidence/docker/* and *evidence/api/screenshots/* are the places to store it before submission.

1. Objectives

Turn a trained classification model into a reproducible service: (1) refactor notebook-style code into a modular pipeline (data, train, evaluate, predict) driven by a configuration file; (2) expose predictions through a validated FastAPI REST API (*POST /predict*, *GET /health*); (3) package the service with Docker (and explore Docker Compose with an MLflow Tracking Server); (4) document everything so another developer can reproduce it, and show that local and containerised predictions agree within a documented tolerance.

Problem and data. Binary classification (benign vs malignant) on scikit-learn's bundled public Breast Cancer Wisconsin dataset: 569 rows, 10 numeric 'mean' features (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension). Public data only; no personal or confidential information. The CSV is versioned with DVC (*data/raw/breast_cancer.csv.dvc*).

2. Architecture



Same Pipeline object is used by training, evaluation, CLI prediction and the API (no training-serving skew)

The inference artefact is a single scikit-learn *Pipeline* (*RatioFeatures* -> median *SimpleImputer* -> *StandardScaler* -> *RandomForestClassifier*), so identical preprocessing is applied in training, evaluation, CLI prediction and the API.

Module	Responsibility	Key functions
src/data.py	Load, validate, clean, feature engineering transformer, split	load_data, validate_schema, preprocess_data, RatioFeatures, split_data
src/train.py	Read config, train, save pipeline + metadata, call evaluation, log to MLflow	load_config, build_pipeline, train_model, run_training, main
src/evaluate.py	Load model + held-out data, metrics, plots, reports	evaluate_model, main
src/predict.py	Load saved pipeline, score records, CLI	load_model, load_metadata, predict, main
src/api.py	FastAPI service, schemas, error handling	/health, /info, /predict, /predict/batch

3. Part A - ML pipeline (Session 7)

3.1 Pipeline stages (A1)

Stage	Input	Output	Purpose	Possible failure
Data loading	CSV path	raw DataFrame	Bring approved data into memory	Missing file / wrong columns
Cleaning	raw DataFrame	clean DataFrame	Drop duplicates, fix formats, flag invalid values	Treating valid zeros as invalid; dropping too many rows
Feature engineering	clean features	model-ready matrix	Ratios, imputation, scaling	Fitting scaler on all data (leakage); column-order skew
Training	X_train, y_train, config	fitted Pipeline	Learn parameters	Bad hyper-parameters; non-reproducible seed
Evaluation	saved model, held-out set	metrics + reports	Measure generalisation	Evaluating on training data
Prediction	saved model, new records	label + probability	Score unseen data	Renamed/missing columns; stale model

Each stage has one responsibility so it can be tested, replaced and reasoned about independently (see docs/CONCEPTS.md).

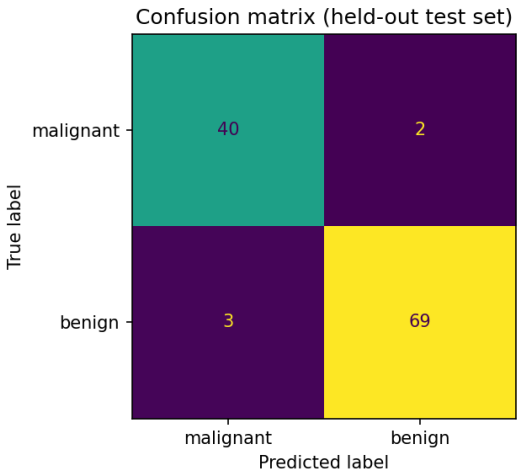
3.2 Configuration and execution (A3)

All changeable values live in *configs/config.yaml* (dataset path, target, feature list, seed 42, test size 0.2, CV folds, model type and hyper-parameters, output paths, MLflow settings). CLI flags (*--config*, *--model-type*, *--no-mlflow*) and the *MLFLOW_TRACKING_URI* environment variable override it.

```
python -m src.train --config configs/config.yaml
python -m src.evaluate
python -m src.predict --sample 5
```

3.3 Results

Metric (114 held-out rows)	Value
Accuracy	0.9561
Precision	0.9718
Recall	0.9583
F1	0.9650
ROC-AUC	0.9917
5-fold CV F1 (training data only)	0.9444 +/- 0.0244
Confusion matrix [[TN,FP],[FN,TP]] (malignant, benign)	[[40, 2], [3, 69]]



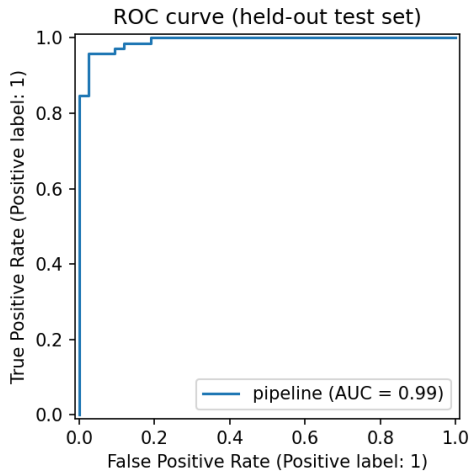


Figure: confusion matrix and ROC curve on the held-out test set (reports/). The model's recall for the malignant class is 0.952 (40/42 detected); in a real clinical setting this metric and the cost of false negatives would need much closer scrutiny - this lab model is a teaching artefact, not a diagnostic tool.

3.4 Validation (A4) VERIFIED

Check	Evidence	Result
Run from terminal	evidence/local/train_log.txt	Completed; model saved, 455 train / 114 test rows
Import has no side effects	test_importing_modules_does_not_start_training	Pass - nothing written on import
No leakage into preprocessing	test_test_data_not_used_to_fit_preprocessing	Pass - scaler mean == train mean (!= full-data mean); train/test disjoint
Load in separate process	test_model_loads_and_predicts_in_a_separate_python_process	Pass - predictions identical to in-process
5 predictions	evidence/local/predict_5_records.txt	5 valid outputs (e.g. malignant 0.9996, benign 0.9965)
Tests	evidence/local/pytest_verbose.txt	28 passed (data 6, pipeline 8, API 14 incl. parametrised cases)

3.5 Experiment tracking

Runs are logged to MLflow (params, metrics, report artefacts and the pipeline as a model). Verified run id: *0bf54084e9024c1ca5b8f35d69db24bf* (experiment *mlops-lab-7-9*, status FINISHED, SQLite backend). Logging is best-effort: a tracking failure never aborts training. The pipeline is logged with cloudpickle because MLflow 3.x's default *skops* format rejected the custom transformer.

4. Part B - FastAPI serving (Session 8)

4.1 REST and inference concepts (B1)

REST exposes operations as HTTP requests/responses; GET reads (health), POST submits data for processing (prediction). The request schema defines required fields and types; the response schema defines the prediction and metadata. **Online inference** answers one or a few records immediately (latency/availability matter); **batch inference** scores a collection or file as a scheduled/asynchronous job (throughput/scheduling matter).

4.2 Application (B2)

Endpoint	Description
GET /health	{"status":"ok","model_loaded":true}
POST /predict	10 required float fields (sizes > 0, shape metrics in [0,1], unknown fields rejected). Returns prediction, label, probability, per-class probabilities, model_version
POST /predict/batch	1-1000 records, same validation (optional extra)
GET /info	Model metadata and MLflow reachability (used to prove API -> MLflow connectivity in Compose)

The model is loaded once at start-up (lifespan handler) from *MODEL_PATH*; the API builds a DataFrame with the exact training column names/order, so the saved Pipeline applies the same preprocessing. A global exception handler returns a generic 500 and logs details server-side only; a missing model gives 503 while */health* stays alive.

4.3 Swagger UI and Postman/curl (B3, B4)

Swagger UI is served at */docs* (generated from the OpenAPI schema, saved as evidence/api/openapi.json). The curl calls below were executed against a live uvicorn process. Swagger UI/Postman screenshots: **RUN ON YOUR MACHINE** (open http://127.0.0.1:8000/docs, try GET /health and POST /predict, save screenshots in evidence/api/screenshots/).

4.4 Validation and error handling (B5) VERIFIED

Test	Expected	Observed (live run)
Valid input	Prediction	HTTP 200: prediction 1 (benign), probability 0.7783
Missing field	422	HTTP 422, 'Field required' for each of the 8 missing fields
Wrong type ("large")	422	HTTP 422 float_parsing on mean_radius
Invalid business value	Controlled error	HTTP 422: radius -5 (must be > 0); smoothness 1.7 (must be <= 1)
Health check	Healthy	HTTP 200 {"status":"ok","model_loaded":true}
Unexpected exception	No leak	HTTP 500 {"detail":"Internal server error."}; service still healthy (test)
Model file missing	Controlled	HTTP 503; /health still 200 (test)

Automated API tests (14, FastAPI TestClient) cover all rows above plus parametrised invalid values, unknown fields, batch limits and API-vs-local equality.

Example exchange (evidence/api/curl_results.txt):

```
 === 1. Health check ===
 $ GET http://127.0.0.1:8000/health
 {"status":"ok","model_loaded":true}
 HTTP_STATUS=200

 === 2. Valid request ===
 $ POST http://127.0.0.1:8000/predict
 {"prediction":1,"label":"benign","probability":0.7783295667033236,"probabilities":{"malignant":0.22167043329667635,"beni
     gn":0.7783295667033236},"model_version":"2026-11-06T13:17:30+00:00"}
 HTTP_STATUS=200
```

4.5 Online versus batch design (B6)

Implemented: small synchronous */predict/batch* and an offline CSV CLI. Proposed for CSV upload: synchronous multipart endpoint for up to a few thousand rows; a background job (202 + job id, worker, status/result endpoints) for tens of thousands; and a separate scheduled batch container reading/writing a store for millions, so heavy scoring never starves the low-latency online API. Details and trade-offs: docs/BATCH_INFERENCE_DESIGN.md.

5. Part C - Docker (Session 9)

5.1 Concepts (C1)

Image: read-only template with code, dependencies and runtime configuration. **Container**: a running instance of an image. **Dockerfile**: build instructions (cached layers). **Port mapping**: *-p host:container*. **Volume**: storage outside the container's writable layer; a host path is not visible inside a container unless mounted. **Environment variable**: runtime configuration without rebuilding. **Compose**: several services, a shared network and volumes in one YAML file.

Why code works on one laptop but not another: different OS, Python/library versions (a pickled scikit-learn model depends on the scikit-learn version), missing system libraries, environment variables, relative paths, files present on only one machine, busy ports. Containers ship interpreter, libraries and files together, so the runtime environment is the same wherever the image runs.

What containers do not guarantee: identical hardware (ARM vs x86, GPU drivers), bit-identical floating-point results across CPUs, code security or correctness, reproducible rebuilds without pinned versions, data persistence without volumes, or resource limits unless configured.

5.2 Implementation (C2-C6)

Item	Implementation
Dockerfile	python:3.11-slim; installs pinned requirements.txt first (layer cache); copies src/, models/, configs/; non-root user; HEALTHCHECK; CMD uvicorn on 0.0.0.0:8000; MODEL_PATH env var
.dockerignore	.git, .venv, __pycache__, *.pyc, data/, reports/, tests/, notebooks, mlruns, .env (no secrets or large data in the image)
Configuration	Env vars MODEL_PATH, MLFLOW_TRACKING_URI, LOG_LEVEL; no credentials baked in
Persistence	Compose: named volume mlflow-data (SQLite DB + artefacts); ./models mounted read-only into the API so a retrained model needs no rebuild
docker-compose.yml	Services api (port 8000), mlflow (ghcr.io/mlflow/mlflow:v3.10.1, port 5000, SQLite backend, --serve-artifacts, --allowed-hosts for the service name) and an optional trainer (profile 'train') that logs to http://mlflow:5000

```
FROM python:3.11-slim

# REQUIREMENTS_FILE lets Compose build a "trainer" image with the extra training deps
ARG REQUIREMENTS_FILE=requirements.txt

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    MODEL_PATH=/app/models/model.joblib \
    LOG_LEVEL=INFO

WORKDIR /app

COPY requirements.txt requirements-train.txt ./
RUN pip install --no-cache-dir -r ${REQUIREMENTS_FILE}

COPY src/ ./src/
COPY models/ ./models/
# configs are tiny and let the same image run `python -m src.train` (trainer service)
COPY configs/ ./configs/

# Run as an unprivileged user
RUN useradd --create-home --uid 10001 appuser && chown -R appuser /app
USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://127.0.0.1:8000/health', ti
    meout=2).status==200 else 1)"

CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Implemented vs proposed. The Dockerfile, .dockerignore, Compose file and validation scripts are implemented and syntactically checked (Compose YAML parses). The Compose file is a single-host design scaffold; the MLflow image tag, flags (including *--allowed-hosts*, required because recent MLflow validates the Host header) and storage layout were taken from current MLflow documentation but have not been executed in

the generation environment, which had no Docker daemon.

5.3 Container validation (C7)

Check	Command	Status
14 Build image	docker build -t ml-inference-api:1.0 .	**RUN ON YOUR MACHINE**
15 Run + /health	docker run -d --name ml-api -p 8000:8000 ml-inference-api:1.0 ; curl localhost:8000/health	**RUN ON YOUR MACHINE**
16 Prediction from host	scripts/api_demo.sh http://localhost:8000	**RUN ON YOUR MACHINE**
17 Logs	docker logs ml-api	**RUN ON YOUR MACHINE**
18 Remove / recreate	docker rm -f ml-api ; docker run ... ; curl /health	**RUN ON YOUR MACHINE**
19 Compose connectivity + persistence	scripts/compose_validate.sh (GET /info shows mlflow_reachable: true; experiments survive down/up)	**RUN ON YOUR MACHINE**

All of C7 and the Part D comparison are produced by one command: *scripts/docker_validate.sh* (writes evidence/docker/docker_validation.txt and reports/local_vs_container.md). Paste the build output, *docker image ls*, *docker logs* and screenshots into Section 5.3 of your final submission.

6. Part D - Final integration

Step	Command	Status
1 Prepare data + train (modular)	python scripts/make_dataset.py ; python -m src.train	**VERIFIED**
2 Evaluate + save pipeline	reports/*, models/model.joblib	**VERIFIED**
3 MLflow tracking	run 0bf54084e902... FINISHED	**VERIFIED**
4 FastAPI + POST /predict	uvicorn src.api:app ; scripts/api_demo.sh	**VERIFIED**
5 Build image	docker build -t ml-inference-api:1.0 .	**RUN ON YOUR MACHINE**
6 Run container + predict from host	docker run ... ; curl /predict	**RUN ON YOUR MACHINE**
7 Teammate reproduction	README section 6	**VERIFIED**

Local vs service consistency. Ten held-out records were scored in-process (*predict()*) and through the running API; classes and probabilities were identical (max |difference| = 0.0, tolerance 1e-6). This verifies the API layer adds no preprocessing drift. The same script run against the container (*--label container*) completes the requirement; tolerance is documented because the same pinned libraries should give identical results, but different CPU/BLAS builds could introduce ~1e-12 differences.

```
 # Local vs api_local predictions

 API: `http://127.0.0.1:8000` | tolerance on probability: 1e-06

 | # | local class | api class | local P | api P | abs diff | match |
 |---|---|---|---|---|---|---|
 | 0 | malignant | malignant | 0.99959789 | 0.99959789 | 0.00e+00 | yes |
 | 1 | benign | benign | 0.99654403 | 0.99654403 | 0.00e+00 | yes |
 | 2 | malignant | malignant | 0.93680229 | 0.93680229 | 0.00e+00 | yes |
 | 3 | malignant | malignant | 0.67733333 | 0.67733333 | 0.00e+00 | yes |
 | 4 | malignant | malignant | 0.99968638 | 0.99968638 | 0.00e+00 | yes |
 | 5 | benign | benign | 0.95472356 | 0.95472356 | 0.00e+00 | yes |
 | 6 | benign | benign | 0.99639179 | 0.99639179 | 0.00e+00 | yes |
 | 7 | malignant | malignant | 0.99959789 | 0.99959789 | 0.00e+00 | yes |
 | 8 | malignant | malignant | 0.99959789 | 0.99959789 | 0.00e+00 | yes |
 | 9 | malignant | malignant | 0.99959789 | 0.99959789 | 0.00e+00 | yes |
```

Result: ALL MATCH (10/10)

7. Reproducing the workflow

```
 git clone <repo-url> && cd mlops-lab-7-9
```

```
python -m venv .venv && source .venv/bin/activate && pip install -r requirements-dev.txt
dvc pull || python scripts/make_dataset.py
python -m src.train && python -m pytest -q
uvicorn src.api:app --port 8000
docker build -t ml-inference-api:1.0 . && docker run -d --name ml-api -p 8001:8000 ml-inference-api:1.0
python scripts/compare_predictions.py --url http://localhost:8001 --label container
```

Versions are pinned (scikit-learn 1.8.0, pandas 3.0.2, FastAPI 0.162.0, ...) because a pickled model must be loaded by the same scikit-learn version; seeds are fixed (42); data is versioned by DVC (MD5 in the .dvc file) and its SHA-256 is stored in model_metadata.json.

8. Challenges and resolutions

Challenge	Resolution
Scaling on all data would leak test information	Imputer + scaler + estimator in one Pipeline fitted on the train split; leakage test
Zero valid for some features, invalid for others	Config list *strictly_positive*
Custom transformer must be importable when unpickling in API/Docker	Defined in src/data.py (not __main__); run via python -m; image copies src/
MLflow 3.x skops rejected the custom transformer	cloudpickle serialisation; tracking wrapped so it cannot break training
Importing modules must not train	All work behind main(); subprocess test
Local HTTP calls went through a proxy in the sandbox	urllib opener with ProxyHandler({}) for localhost
Errors must not leak internals	Typed validation (422), 503, generic 500 + server-side logging
MLflow validates Host header	--allowed-hosts including the Compose service name
No Docker daemon in the generation environment	Scripted validation + explicit VERIFIED / RUN-ON-YOUR-MACHINE labelling

9. Reflection

Packaging preprocessing with the estimator removed the most common source of production bugs (training-serving skew), and writing the leakage and separate-process tests turned 'it works in my notebook' into checkable claims. Strict request schemas moved most input errors to the edge of the system as clear 422 responses, and keeping the 500 handler generic protects internals. Docker adds reproducibility of the environment, not of the model's correctness or of hardware-level numerics, which is why a numerical tolerance is documented. Limitations: the dataset is small and public, the model is not clinically validated, pickle files are only safe from trusted sources, the Compose design is single-host and unauthenticated, and no load/latency testing was done. Next steps: CI to run tests and build the image, a model registry with promotion, authentication and rate limiting, and an asynchronous batch worker.